



Natural Language Processing: RLHF

HSE Faculty of Computer Science
Machine Learning and Data-Intensive Systems

Murat Khazhgeriev



Table of Content

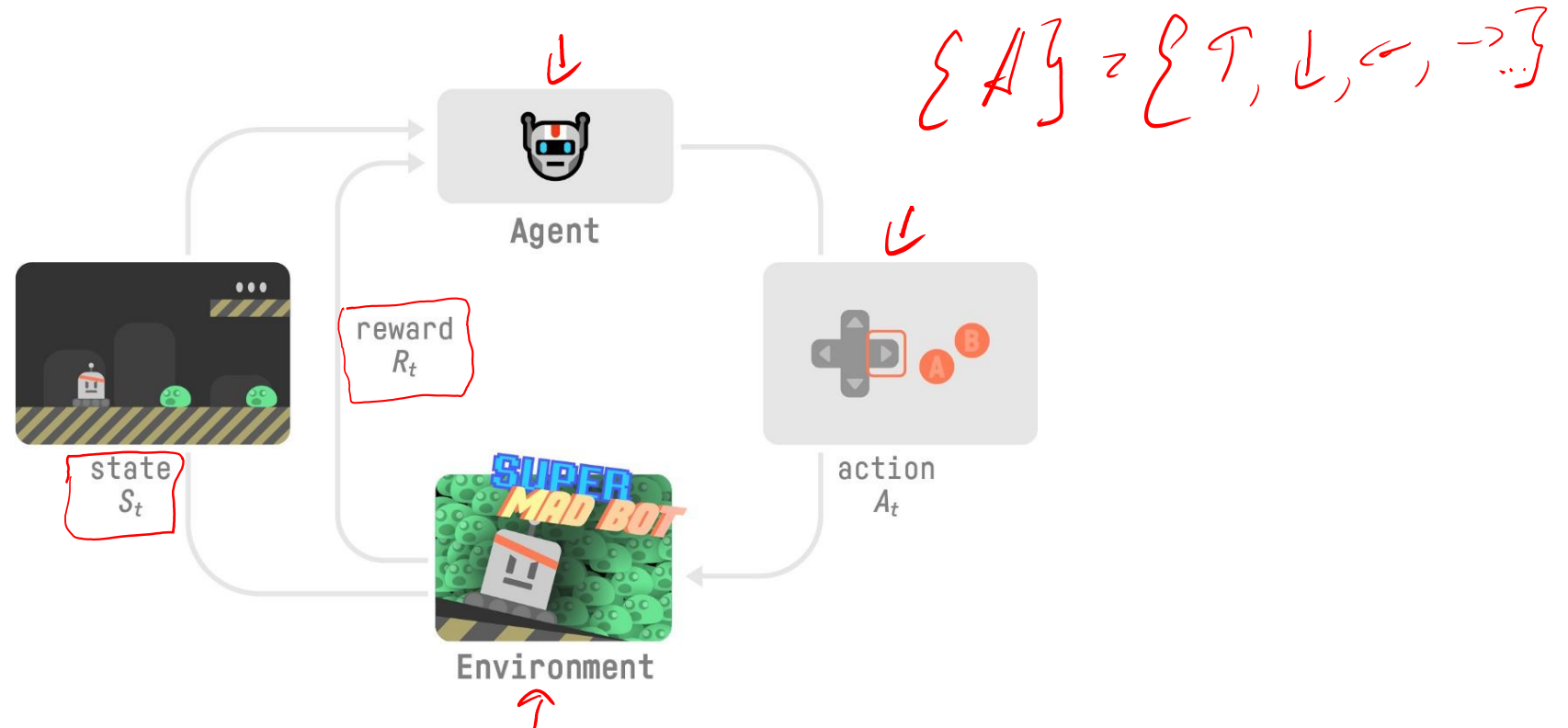
- Intro to RL
- RLHF pipeline
- PPO
- DPO



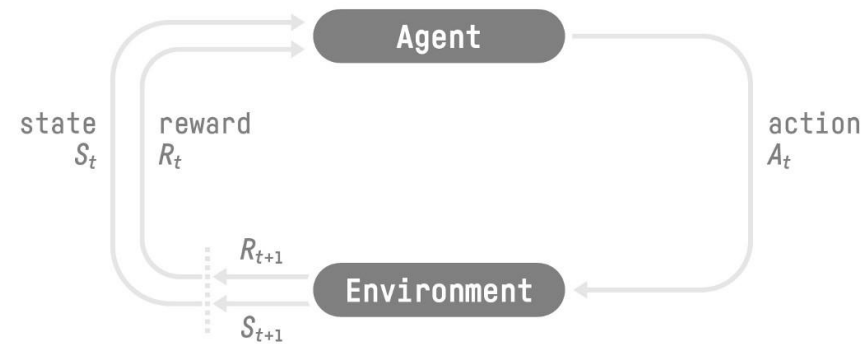
Table of Content

- **Intro to RL**
- RLHF pipeline
- PPO
- DPO

An Agent acts upon an Environment, changes the State and gets a Reward



An Agent acts upon an Environment, changes the State and gets a Reward



Where to learn more?

- [Deep RL Course by HF](#)
- [YSDA Practical RL](#)



Table of Content

- Intro to RL
- **RLHF pipeline**
- PPO
- DPO

RLHF is implemented in 3 main steps: SFT, RM, RL

Pre-train

t_1, t_2, t_3
↑ ↑ ↑
[LCM] ←

$t_0, t_1, t_2, \dots$

Alignment

SFT

bounded offset
~~~~~|~~~~~  
~~~~~|~~~~~

Step 1

Collect demonstration data, and train a supervised policy.

A prompt is sampled from our prompt dataset.

Explain the moon landing to a 6 year old

A labeler demonstrates the desired output behavior.

Some people went to the moon...

This data is used to fine-tune GPT-3 with supervised learning.

SFT
RM

Step 2

Collect comparison data, and train a reward model.

A prompt and several model outputs are sampled.

Explain the moon landing to a 6 year old

A Explain gravity... B Explain war...

C Moon is natural satellite of... D People went to the moon...

A labeler ranks the outputs from best to worst.

D > C > A = B

This data is used to train our reward model.

RM
D > C > A = B

Step 3

Optimize a policy against the reward model using reinforcement learning.

A new prompt is sampled from the dataset.

Write a story about frogs

The policy generates an output.

PPO

Once upon a time...

The reward model calculates a reward for the output.

RM

The reward is used to update the policy using PPO.

r_k

$V_{\pi} = z(y_w) - z(y_e)$

$z(y_w) > z(y_e)$

RLHF is implemented in 3 main steps: SFT, RM, RL

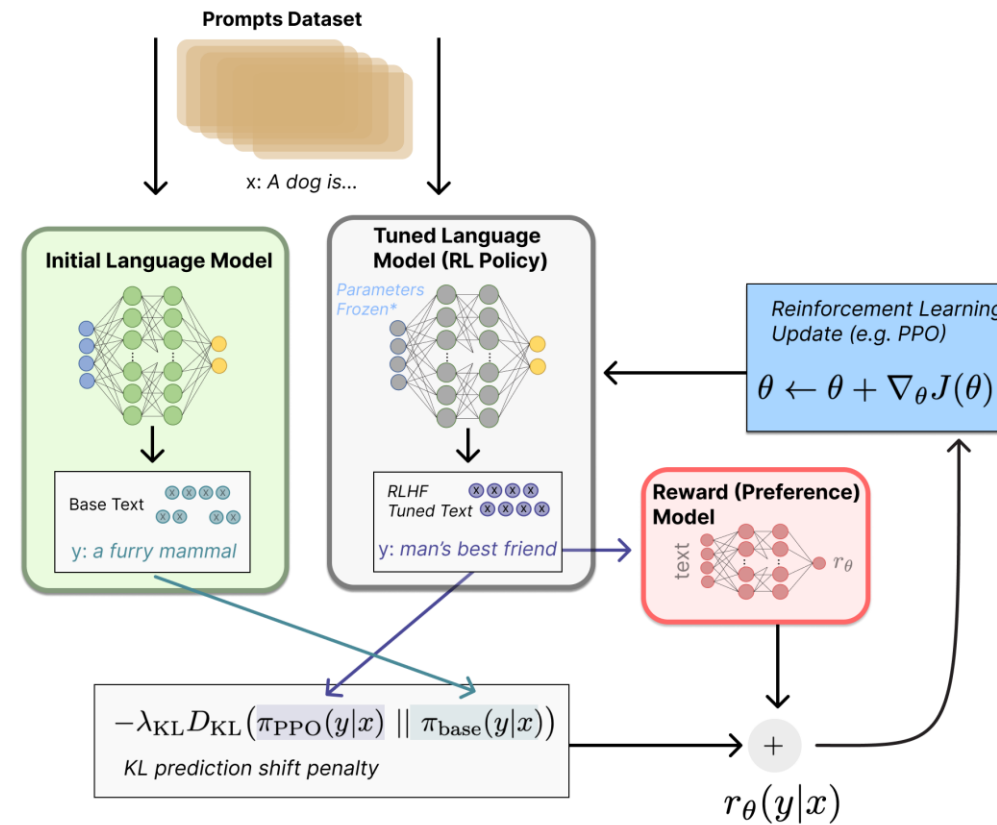




Table of Content

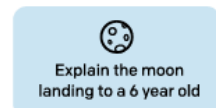
- Intro to RL
- RLHF pipeline
- **PPO**
- DPO

Step 1: Train an initial model with Supervised Finetuning

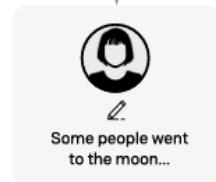
Step 1

Collect demonstration data, and train a supervised policy.

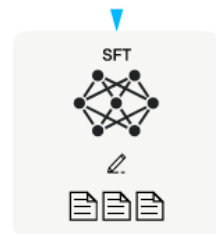
A prompt is sampled from our prompt dataset.



A labeler demonstrates the desired output behavior.



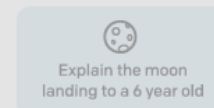
This data is used to fine-tune GPT-3 with supervised learning.



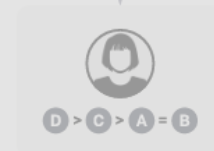
Step 2

Collect comparison data, and train a reward model.

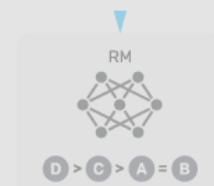
A prompt and several model outputs are sampled.



A labeler ranks the outputs from best to worst.



This data is used to train our reward model.



Step 3

Optimize a policy against the reward model using reinforcement learning.

A new prompt is sampled from the dataset.



The policy generates an output.

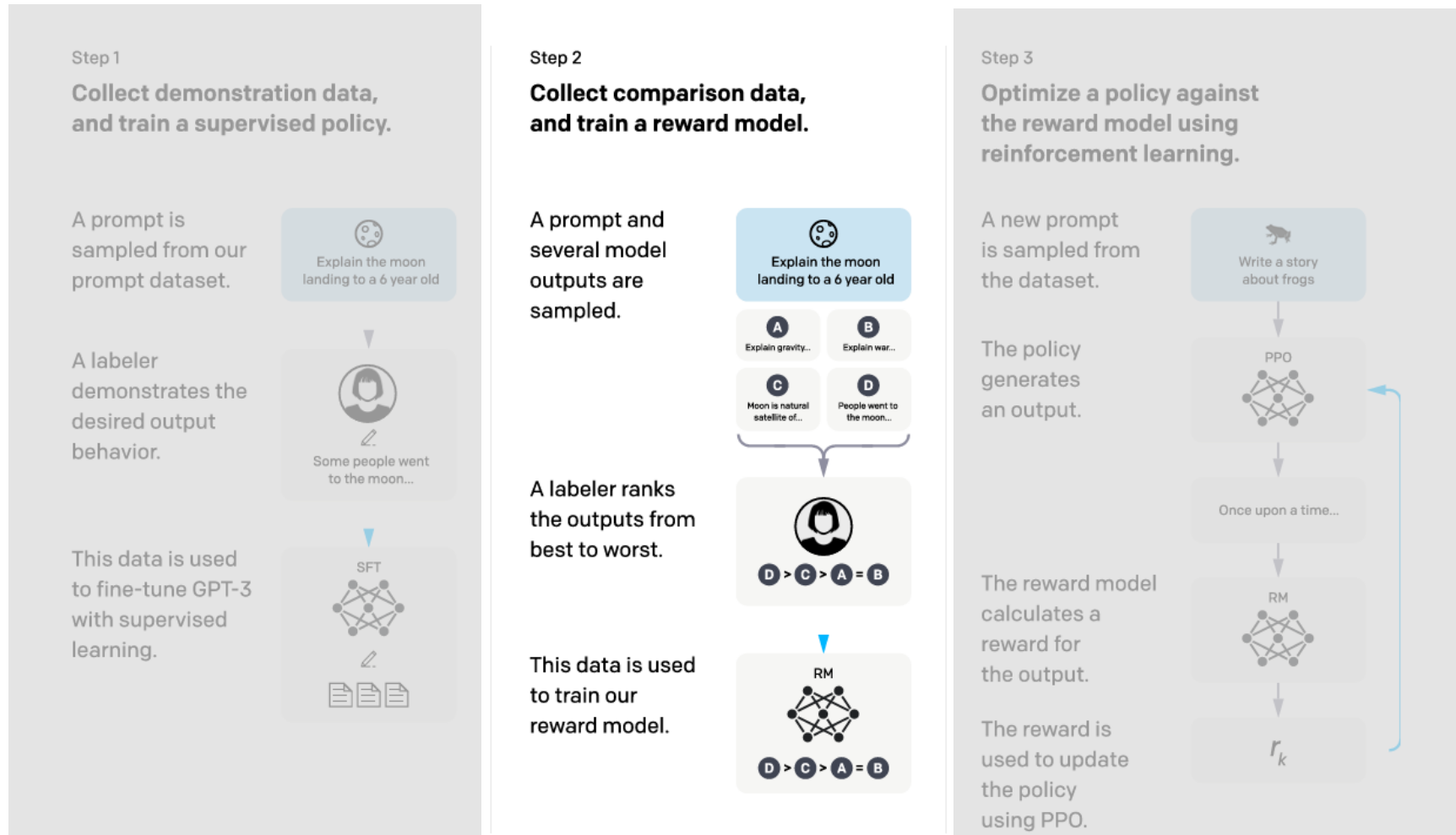


The reward model calculates a reward for the output.



The reward is used to update the policy using PPO.

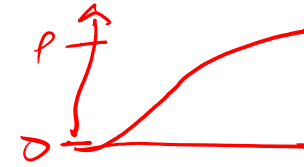
Step 2: Train a Reward Model



Reward Model learns to differentiate between winner's and loser's scores

$y_0, y_1, \dots, y_k$
 $\underbrace{}_{\text{LCM}}$
 $\times$

$z(y_i)$
 $\underbrace{}_{\text{RM}}$
 $\times$



$$\text{loss}(\theta) = -\frac{1}{\binom{K}{2}} E_{(x, y_w, y_l) \sim D} [\log(\sigma(r_\theta(x, y_w) - r_\theta(x, y_l)))]$$

$$C_k^2 = \frac{k(k-1)}{2!} =$$

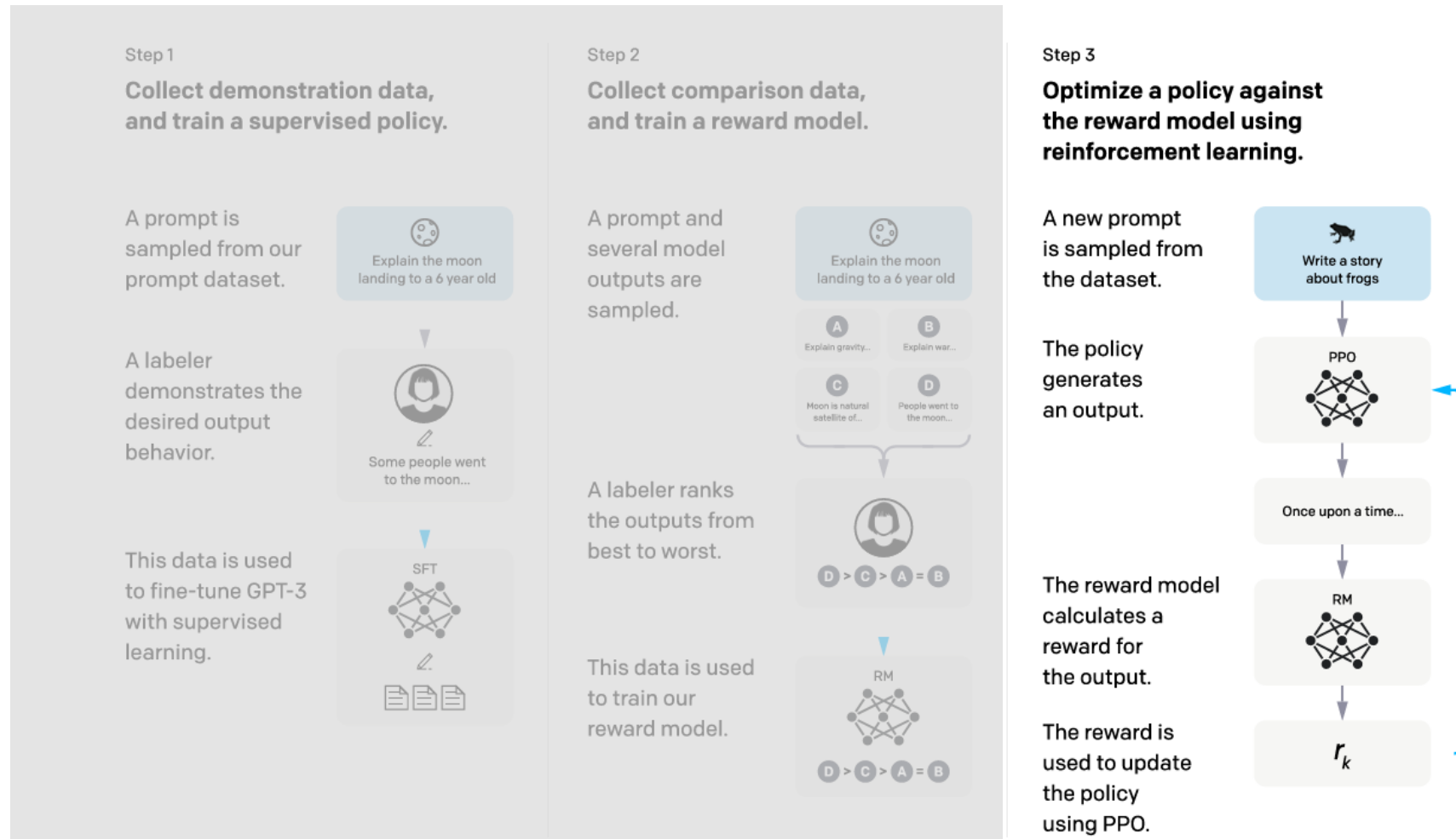
$$= \frac{k!}{(k-2)!2!} = \binom{k}{2}$$

Reward Model learns to differentiate between winner's and loser's scores

$$\text{loss}(\theta) = -\frac{1}{\binom{K}{2}} E_{(x, y_w, y_l) \sim D} [\log(\sigma(r_\theta(x, y_w) - r_\theta(x, y_l)))]$$

...see the whiteboard for more!

Step 3: Introduce Reinforcement Learning to the problem



Leverage a vanilla Proximal Policy Optimization setup

ppo

$$\text{objective}(\phi) = E_{(x,y) \sim D_{\pi_{\phi}^{\text{RL}}}} \left[r_{\theta}(x, y) - \underbrace{\beta \log \left(\pi_{\phi}^{\text{RL}}(y | x) / \pi^{\text{SFT}}(y | x) \right)}_{\text{KL divergence}} \right]$$

Add regularization via pretrain tasks to prevent deterioration

$$\text{objective}(\phi) = \underbrace{E_{(x,y) \sim D_{\pi_{\phi}^{\text{RL}}}} \left[r_{\theta}(x, y) - \beta \log \left(\pi_{\phi}^{\text{RL}}(y | x) / \pi^{\text{SFT}}(y | x) \right) \right]}_{\text{RLHF loss}} + \underbrace{\gamma E_{x \sim D_{\text{pretrain}}} \left[\log(\pi_{\phi}^{\text{RL}}(x)) \right]}_{\text{pretrain loss}}$$

...see the whiteboard for more!



Table of Content

- Intro to RL
- RLHF pipeline
- PPO
- **DPO**

Your Language Model is Secretly a Reward Model

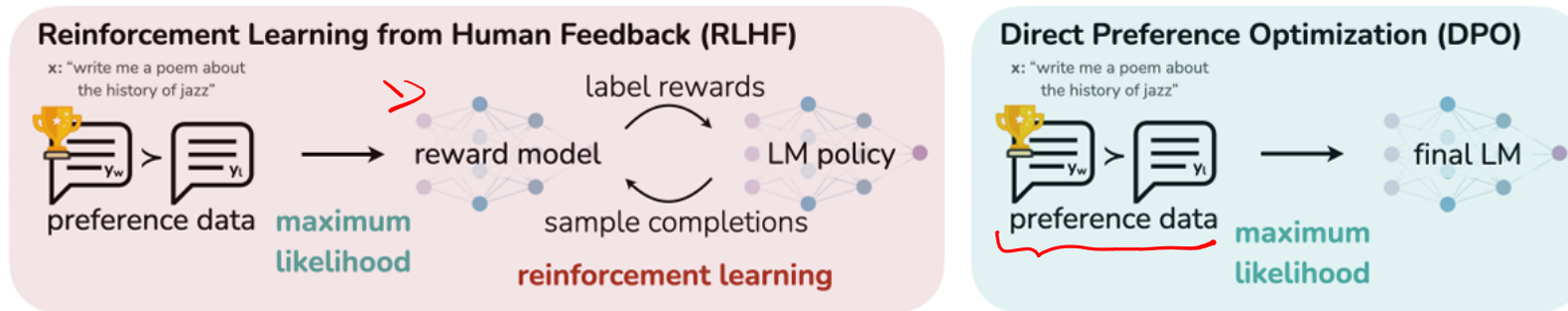


Figure 1: **DPO optimizes for human preferences while avoiding reinforcement learning.** Existing methods for fine-tuning language models with human feedback first fit a reward model to a dataset of prompts and human preferences over pairs of responses, and then use RL to find a policy that maximizes the learned reward. In contrast, DPO directly optimizes for the policy best satisfying the preferences with a simple classification objective, fitting an *implicit* reward model whose corresponding optimal policy can be extracted in closed form.

Your Language Model is Secretly a Reward Model

DPO
Direct Preference Optimisation

$$\mathcal{L}_{\text{DPO}}(\pi_{\theta}; \pi_{\text{ref}}) = -\mathbb{E}_{(x, y_w, y_l) \sim \mathcal{D}} \left[\log \sigma \left(\beta \log \frac{\pi_{\theta}(y_w | x)}{\pi_{\text{ref}}(y_w | x)} - \beta \log \frac{\pi_{\theta}(y_l | x)}{\pi_{\text{ref}}(y_l | x)} \right) \right]$$

Your Language Model is Secretly a Reward Model

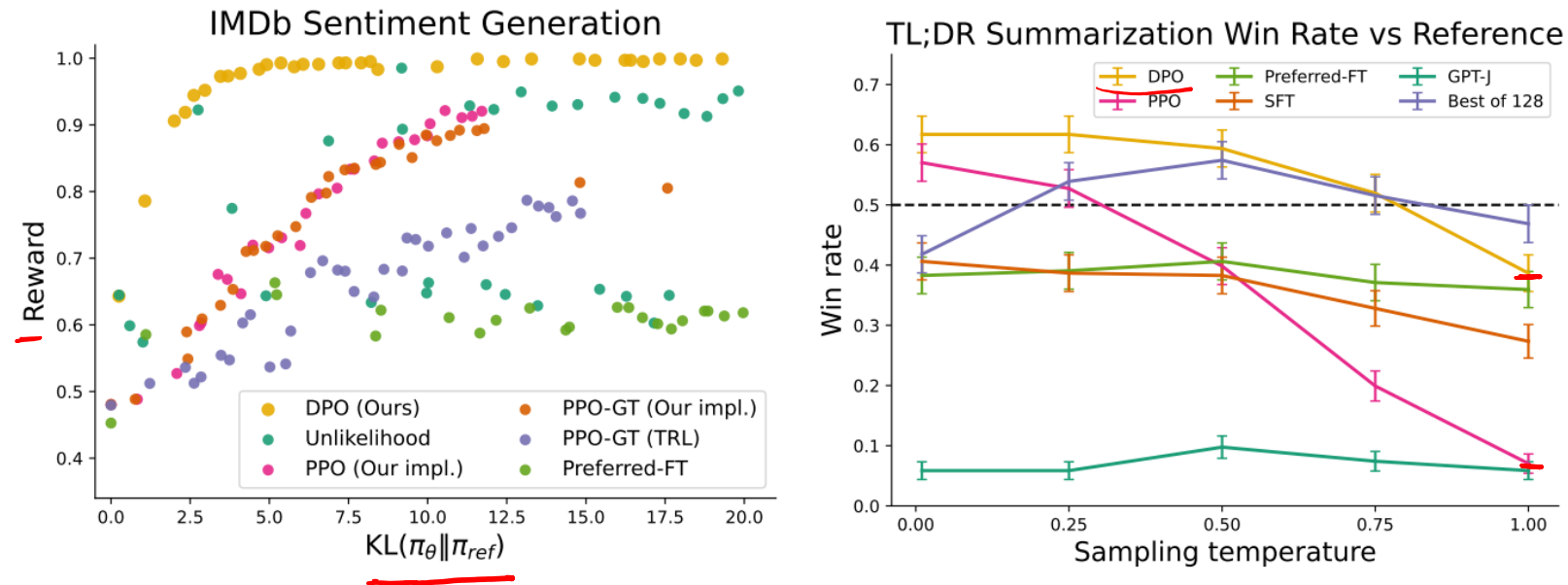


Figure 2: **Left.** The frontier of expected reward vs KL to the reference policy. DPO provides the highest expected reward for all KL values, demonstrating the quality of the optimization. **Right.** TL;DR summarization win rates vs. human-written summaries, using GPT-4 as evaluator. DPO exceeds PPO's best-case performance on summarization, while being more robust to changes in the sampling temperature.

